



Tehnologii WEB

Curs 8-9

PHP

Cod Teams: **6gyjd1x**

Anul III, semestrul II

2026

PHP

Cuprins:

- Definitii & Istoric
- Vocabular & Sintaxa
- Tipuri de date
- Variabile & Constante
- Operatori
- Instructiuni conditionale
- Functii
- Exemple rezolvate complet



Definitii, Istoric si Caracteristici PHP

Contextul Web — Client si Server

PHP nu poate fi inteles fara sa stii cum functioneaza un site web. Exista doua entitati care comunica in permanenta:

CLIENT (browser)

Chrome, Firefox, Safari, Edge
Interpreteaza HTML, CSS, JavaScript
Ruleaza pe calculatorul utilizatorului
Trimite cereri HTTP (REQUEST)
Primeste si afiseaza HTML (RESPONSE)
NU stie nimic despre PHP

SERVER (Apache / Nginx)

Ruleaza PHP, Python, Node.js etc.
Acceseaza baze de date (MySQL)
Citeste / scrie fisiere pe disc
Primeste cereri HTTP (REQUEST)
Executa PHP si trimite HTML (RESPONSE)
Browserul NU vede niciodata codul PHP

Fluxul complet al unei cereri web — pas cu pas:

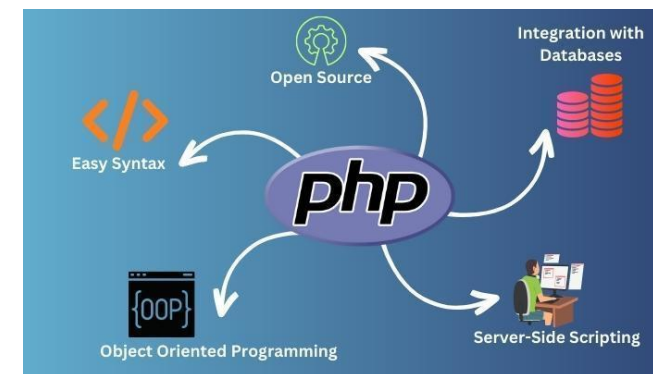
- a) Utilizatorul tasteaza `www.exemplu.ro` in browser si apasa Enter
- b) Browserul face o cerere HTTP GET catre serverul `www.exemplu.ro`
- c) Serverul primeste cererea si identifica fisierul `index.php`
- d) Serverul porneste interpretorul PHP care executa scriptul
- e) PHP acceseaza baza de date, citeste fisiere, calculeaza — produce HTML pur
- f) Serverul trimite HTML-ul catre browser (browserul NU primeste cod PHP)
- g) Browserul interpreteaza HTML-ul si afiseaza pagina utilizatorului

PHP = limbaj EXCLUSIV server-side. Codul PHP niciodata nu ajunge la browser.

Browserul primeste mereu HTML pur, indiferent ce limbaj a generat acel HTML.

Tocmai de aceea PHP poate accesa baze de date si fisiere fara riscuri de securitate directe.

PHP (acronim recursiv: PHP: Hypertext Preprocessor) este un limbaj de programare de tip scripting, general-purpose cu accent pe web, open-source, creat special pentru generarea dinamica de pagini HTML pe server.



PHP este un limbaj de scripting interpretat, server-side, cu tipizare dinamica, incorporabil direct in HTML, proiectat pentru dezvoltarea rapida a aplicatiilor web.

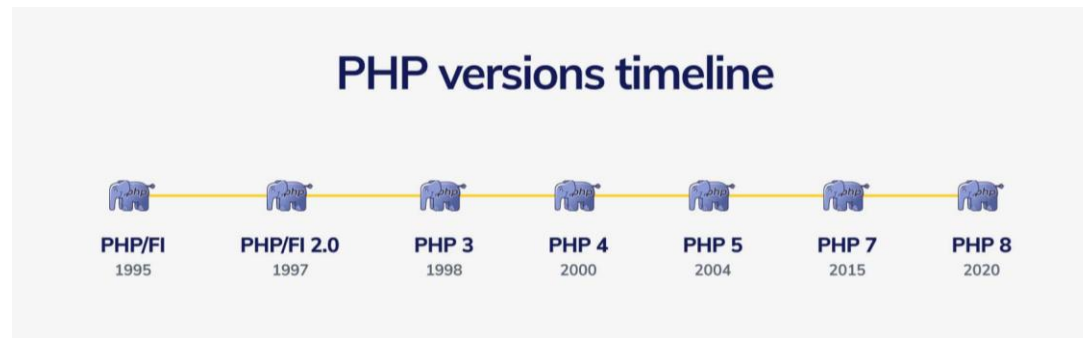
Codul PHP este executat pe server; rezultatul (de obicei HTML) este transmis clientului.

Versiunea curenta stabila: **PHP 8.3 (2023)**

Licenta: PHP License (compatibila GPL, open-source)

Statistici relevante (2025):

- Peste 77% din toate site-urile web cu limbaj de server cunoscut folosesc PHP
- WordPress (43% din internet), Facebook (partial), Wikipedia — toate folosesc PHP
- Peste 5.000 de functii built-in disponibile
- Ecosistem vast: Composer, Laravel, Symfony, CodeIgniter, Wordpress



*php*8

An	Versiune	Inovatie principala	Cine / Context
1994	PHP/FI 1.0	Rasmus Lerdorf creeaza scripturi CGI in C pentru a urmari vizitele pe CV-ul sau online. Le numeste Personal Home Page Tools.	Rasmus Lerdorf (canada)
1997	PHP 3.0	Zeev Suraski si Andi Gutmans rescriu complet motorul. Noul acronim PHP: Hypertext Preprocessor. Prima versiune larg adoptata.	Tel Aviv University
2000	PHP 4.0	Zend Engine 1.0. Performante mult imbunatatite. Suport pentru sesiuni si multe extensii noi. Adoptie masiva.	Zend Technologies
2004	PHP 5.0	Zend Engine 2.0. OOP complet (clase, interfete, exceptii). PDO, SimpleXML. Saltul calitativ major.	Comunitatea PHP
2015	PHP 7.0	Zend Engine 3.0. Viteza 2x fata de PHP 5.6. Tipuri scalare, operatorul ??, operatorul spaceship <=>.	PHP Core Team
2016	PHP 7.1	Nullable types (?int), void return, list() in foreach, multi-catch.	PHP Core Team
2019	PHP 7.4	Typed properties, arrow functions (fn=>), preloading, FFI.	PHP Core Team
2020	PHP 8.0	JIT Compiler, Union Types, Named Arguments, Attributes (anotari native), Match expression, Nullsafe operator ?->.	PHP Core Team
2021	PHP 8.1	Fibers, Readonly properties, Enums, Intersection types, never return type.	PHP Core Team
2022	PHP 8.2	Readonly classes, DNF types, true/false/null ca tipuri standalone.	PHP Core Team
2023	PHP 8.3	Typed class constants, json_validate(), override attribute, granular DateTime exceptions. Versiunea actuala.	PHP Core Team

Caracteristici Esentiale PHP

A) Open-source si cross-platform

PHP este gratuit, cu sursa deschisa. Ruleaza identic pe Linux, Windows si macOS, pe orice server web (Apache, Nginx, IIS). Nu necesita licenta si are o comunitate imensa de dezvoltatori.

B) Embedded in HTML — cod PHP direct in pagina web

Codul PHP se poate intercala direct in HTML, fara fisiere separate:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Bun venit!</h1>
  <p>
    <?php
      $ora = date('H');
      if ($ora < 12) {
        echo "Buna dimineata!";
      } elseif ($ora < 18) {
        echo "Buna ziua!";
      } else {
        echo "Buna seara!";
      }
    ?>
  </p>
  <p>Data curenta: <?= date('d.m.Y') ?></p>
</body>
</html>
```

>> OUTPUT:

```
Buna ziua!          (daca e ora 14:00)
Data curenta: 17.04.2025
```

Caracteristici (continua)

C) Tipizare dinamica — tipul variabilei e determinat automat

```
// PHP determina tipul automat
$x = 42;           // integer
$x = "patruzeci"; // acum e string — PHP nu se plange!
$x = 3.14;         // acum e float
$x = true;         // acum e boolean

// PHP 8 — tipizare stricta optionala
declare(strict_types=1);
function aduna(int $a, int $b): int {
    return $a + $b;
}
aduna(2, 3);      // OK: 5
aduna(2, "3");    // EROARE in strict mode!
```

D) Suport nativ pentru baze de date (MySQL, PostgreSQL, SQLite)

```
// Conectare la MySQL in 3 linii
$db = new PDO("mysql:host=localhost;dbname=studenti", "user",
"parola");
$stmt = $db->query("SELECT * FROM studenti WHERE medie > 8");
foreach ($stmt as $rand) { echo $rand['nume'] . "\n"; }
```

- *PHP suporta: MySQL, PostgreSQL, SQLite, MongoDB, Oracle, MSSQL si altele.*

Configurarea Mediului de Dezvoltare

Pentru a rula PHP local aveti nevoie de un server web cu PHP.

Pachet recomandat: XAMPP (Windows / Mac / Linux)

1. Descarca de la: <https://www.apachefriends.org>
2. Instaleaza si porneste modulele **Apache** si **MySQL**
3. Pune fisierele .php in: **C:\xampp\htdocs\ (Win)** sau **/opt/lampp/htdocs/ (Linux)**
4. Acceseaza in browser: **http://localhost/fisier.php**



```
// Test instalare PHP - salveaza ca: test.php in htdocs/  
<?php  
phpinfo();    // afiseaza TOATE detaliile instalarii PHP  
?>  
  
// Sau mai simplu:  
<?php  
echo "PHP " . PHP_VERSION . " functioneaza corect!";  
echo "<br>Server: " . $_SERVER['SERVER_SOFTWARE'];  
?>
```

>> OUTPUT:

```
PHP 8.3.4 functioneaza corect!  
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.3.4
```

Alternative: Laragon (Windows), MAMP (Mac), Docker, Herd (Laravel)

Vocabularul PHP — Delimitatori

Vocabularul unui limbaj de programare cuprinde toate simbolurile si cuvintele cu semnificatie speciala pe care interpretorul le recunoaste. In PHP avem urmatoarele categorii:

Delimitatorii PHP

Codul PHP trebuie inclus intre delimitatori pentru a fi recunoscut de server:

```
<?php ?>      // Forma standard (INTOTDEAUNA folosita)
<?=    ?>      // Scurtura pentru echo – doar pentru afisare simpla
// <? ?> (scurta) – EVITATA, dezactivata pe multe servere

// Example:
<?php echo 'Hello World'; ?>
<?= 'Hello World' ?>    // identic cu de mai sus

// Fisier doar PHP (fara HTML) – fara tag de inchidere ?>
<?php
// Best practice: nu pune ?> la finalul fisierelor .php pure
// Evita spatii/newlines accidentale inainte de headers HTTP
```

Cuvinte Cheie Rezervate (Keywords)

Nu pot fi folosite ca nume de variabile, functii sau clase:

// Structuri de control

```
if   elsif   else   switch   case   default   break   continue  
for   foreach   while   do   match   return   yield   goto
```

// Declaratii

```
function   class   interface   trait   abstract   final  
extends   implements   namespace   use   const
```

// Tipuri si valori

```
true   false   null  
int     float   string   bool    array   object   void    never  
mixed   callable   iterable   self   static   parent
```

// Altele

```
echo   print   isset   empty   unset   list   new   clone   instanceof  
include   require   include_once   require_once  
try   catch   finally   throw
```

Identificatorii — Reguli de Denumire

Identificatorii sunt numele pe care le dam variabilelor, functiilor, claselor si constantelor. Reguli pentru variabile, functii, clase si constante:

- Incep cu litera (a-z, A-Z) sau underscore (_) — NICIODATA cu cifra
- Pot contine litere, cifre (0-9) si underscore
- **CASE-SENSITIVE: \$Varsta, \$varsta si \$VARSTA sunt variabile DIFERITE**
- Variabilele TREBUIE sa inceapa cu \$
- Conventie: snake_case pentru variabile/functii, PascalCase pentru clase

```
// Identificatori VALIDI
$varsta           // simplu, snake_case
$numeStudent      // camelCase (folosit mai rar in PHP)
$nota_finala      // snake_case (RECOMANDAT in PHP)
$_variabila_interna // conventie pentru variabile 'private'
$variabila2        // cifra in mijloc sau la sfarsit — OK
$CamelCase         // valid dar unconventional pentru variabile

// Identificatori INVALIDI — vor produce erori de sintaxa
// $2variabila     → incepe cu cifra
// $var-buna       → cratima nu este permisa
// $class          → cuvant cheie rezervat
// $un spatiu      → spatiul nu e permis
// varsta          → lipseste $
```

Identificatorii

Exercitiu 1.1 — Identificatori corecti vs incorecti

Cod PHP

```
<?php
$_log = "server.log";    // VALID
$nota_1 = 9.5;           // VALID
$MAX = 100;              // VALID
// $2fast = 'nu';        // INVALID
// $my-var = 1;          // INVALID

var_dump($_log);
var_dump($nota_1);
var_dump($MAX);
?>
```

Output in browser

```
string(10) "server.log"
float(9.5)
int(100)
```

Nota: var_dump()
afiseaza tipul si valoarea
— indispensabil pentru
debugging

Comentarii — Ignorate de Interpretorul PHP

Comentariile nu ajung niciodata in output sau in browser:

```
// Comentariu pe o linie – stilul C++ (cel mai frecvent folosit)

# Comentariu pe o linie – stilul bash/shell (mai rar)

/*
    Comentariu pe mai multe linii.
    Folosit pentru blocuri mari de explicatii.
    Poate contine orice text.
*/

/**
 * PHPDoc Block – documentare formala (standardul in proiecte serioase)
 * @param int    $n    Numarul de elemente
 * @param string $tip   Tipul de sortare: 'asc' sau 'desc'
 * @return array   Array-ul sortat
 * @throws InvalidArgumentException Daca tipul e invalid
 */
function sorteaza(int $n, string $tip): array { }
```

Expresii vs. Instructiuni

Aceasta distinctie este fundamentala in PHP si in orice limbaj de programare:

EXPRESIE

Orice constructie care produce o valoare.

In PHP, aproape totul este o expresie.

<code>5</code>	→ valoarea 5
<code>\$x</code>	→ valoarea lui <code>\$x</code>
<code>\$x + 3</code>	→ suma
<code>strlen('PHP')</code>	→ 3
<code>\$x = 10</code> (produce 10)	→ si aceasta e expresie
<code>\$x > 5</code>	→ true sau false

INSTRUCTIUNE

O unitate de executie completa.

Se termina INTOTDEAUNA cu ; (punct si virgula).

```
$x = 5;           // asignare
echo "val: " . $x; // afisare
$y = $x * 2 + 1;  // calcul
if ($x > 3) { ... } // conditionala
for ($i=0; $i<5; $i++) {} // bucla
```

!! ATENTIE / EROARE FRECVENTA:

// Eroare frecventa: uitarea punct-virgulei

```
$x = 5    // EROARE: Parse error: syntax error, unexpected ...
```

// Exceptie: dupa acolade de inchidere NU se pune ;

```
if ($x > 0) { echo 'ok'; }    // corect - fara ; dupa }
```

```
function f() { return 1; }    // corect - fara ; dupa }
```

Tipuri de Date in PHP

PHP are 8 tipuri de date primitive, grupate in 3 categorii. Tipul se determina dinamic la runtime, in functie de valoarea asignata.

Tipuri scalare (o singura valoare)

Tipul integer (int): Numere intregi, pozitive sau negative, fara parte fractionara. Limita: `PHP_INT_MAX` = 9.223.372.036.854.775.807 (pe sisteme pe 64-bit).

```
// Notatie zecimala (baza 10) - forma obisnuita
$a = 42;
$b = -17;
$c = 0;

// Notatie hexazecimala (baza 16) - prefix 0x
$hex = 0xFF;      // = 255 in zecimal
$hex2 = 0x1A;     // = 26 in zecimal
// Notatie octala (baza 8) - prefix 0
$oct = 0777;      // = 511 in zecimal
// Notatie binara (baza 2) - prefix 0b
$bin = 0b11001;   // = 25 in zecimal

// Separator de mii (PHP 7.4+) - underscore pentru lizibilitate
$populatie = 19_000_000; // 19 milioane
$viteza_lumina = 299_792_458;

// Afisare si verificare
echo $hex;          // 255
echo PHP_INT_MAX;   // 9223372036854775807
echo PHP_INT_SIZE;  // 8 (bytes, pe 64-bit)
var_dump(42);       // int(42)
```

>> OUTPUT:

```
255
9223372036854775807
8
int(42)
```


Exercitiu 2.1 — Operatii cu intregi

Cod PHP

```
<?php
$a = 17;
$b = 5;

echo $a + $b . "\n";    // adunare
echo $a - $b . "\n";    // scadere
echo $a * $b . "\n";    // inmultire
echo intdiv($a, $b) . "\n"; // impartire intreaga
echo $a % $b . "\n";    // modulo (rest)
echo $a ** $b . "\n";   // putere

// Test limita
echo PHP_INT_MAX + 1 . "\n"; // overflow → float!
?>
```

Output in browser

```
22
12
85
3
2
1419857
9.2233720368548E+18  (a
devenit float!)
```

Nota: intdiv() face impartire intreaga. Operatorul % da restul. La depasire, PHP converteste automat la float.

Tipul float (numar cu virgula mobila)

Numere reale, IEEE 754 double precision (64-bit). Alias: double.

```
$pi    = 3.14159265358979;
$g     = 9.81;           // acceleratia gravitationala
$temp  = -23.5;

// Notatie stiintifica
$avogadro = 6.022e23;    // 6.022 * 10^23
$selectron = 1.6e-19;    // 1.6 * 10^-19

// Constante matematice predefinite
echo M_PI;              // 3.1415926535898
echo M_E;               // 2.718281828...
echo M_SQRT2;           // 1.4142135623...

// ATENTIE la precizie - eroare clasica de floating point!
var_dump(0.1 + 0.2);     // float(0.30000000000000004) ← NU 0.3!
var_dump(0.1 + 0.2 == 0.3); // bool(false) ← surpriza!

// Solutie corecta - comparatie cu epsilon
$epsilon = 1e-10;
var_dump(abs((0.1 + 0.2) - 0.3) < $epsilon); // bool(true)

// Rotunjire pentru afisare
echo round(3.14159, 2);  // 3.14
echo number_format(1234567.891, 2, ',', '.'); // 1.234.567,89
```

>> OUTPUT:

```
float(0.30000000000000002)    ← eroare de reprezentare binara
bool(false)                  ← 0.1+0.2 nu este egal cu 0.3!
bool(true)                   ← comparatia cu epsilon e corecta
3.14
1.234.567,89
```

Tipul string ((sir de caractere)

Un string este o secventa de caractere. PHP nu are tip char separat — un caracter e un string de lungime 1.

```
// --- 1. GHILIMELE DUBLE (") --- interpreteaza variabile si escape sequences
$prenume = "Ana";
$familie = "Ionescu";
echo "Studentul $prenume $familie\n";    // variabilele sunt inlocuite
echo "Tab:\there\n";                    // \t = tab, \n = newline
echo "Ghilimele: \"citat\"\n";          // \" pentru ghilimele in sir

// Expresie complexa in ghilimele duble → folosim {}
$note = [9, 8, 10];
echo "Prima nota: {$note[0]}\n";        // 9
// --- 2. GHILIMELE SIMPLE (') --- text LITERAL, fara interpretare
echo 'Studentul $prenume $familie\n';   // afiseaza LITERAL $prenume etc.
echo 'Singurul escape: \' si \\\';      // doar \' si \\ functioneaza
// --- 3. HEREDOC --- string multiline, cu interpretare variabile
$medie = 8.75;
$raport = <<<EOT
Raport student:
    Nume: {$prenume} {$familie}
    Medie: {$medie}
    Status: Admis
EOT;
echo $raport;
// --- 4. NOWDOC --- ca heredoc dar fara interpretare (ca ghilimele simple)
$literal = <<<'EOT'
Variabila: $prenume nu e inlocuita
EOT;
echo $literal;
```

>> OUTPUT:

```
Studentul Ana Ionescu
Tab:   here
Ghilimele: "citat"
Prima nota: 9
Studentul $prenume $familie\n    (literal!)
Raport student:
    Nume: Ana Ionescu
    Medie: 8.75
    Status: Admis
Variabila: $prenume nu e inlocuita
```

String — Operatii Esentiale

```
// Concatenare – operatorul . (punct)
$a = "Automatica";
$b = " si Informatica";
echo $a . $b;                                // "Automatica si Informatica"

// Acces la caracter individual – ca un array
$sir = "PHP";
echo $sir[0];      // "P"
echo $sir[1];      // "H"
echo $sir[2];      // "P"
echo $sir[-1];     // "P" (ultimul caracter, PHP 7.1+)

// Lungime
echo strlen('Salut');    // 5

// Cautare si inlocuire
echo strpos("PHP e bun", "bun");    // 6 (pozitia de inceput)
echo str_replace("PHP", "Python", "PHP e grozav");    // Python e grozav

// Taiere si extragere
echo substr("Automatica", 0, 4);    // "Auto"
echo substr("Automatica", -4);      // "tica"

// Transformare
echo strtoupper("php");    // "PHP"
echo strtolower("PHP");    // "php"
echo ucfirst("ion");       // "Ion"
echo ucwords("ion popescu"); // "Ion Popescu"
echo trim("  hello ");    // "hello" (elimina spatii)

// Verificari PHP 8+
var_dump(str_contains("PHP e bun", "bun"));    // bool(true)
var_dump(str_starts_with("PHP e bun", "PHP")); // bool(true)
var_dump(str_ends_with("PHP e bun", "bun"));   // bool(true)
```

Tipul boolean — Valori Falsy si Truthy

Valorile true si false (case-insensitive). Fundamentale in conditii si comparatii.

```
$logat = true;
$admin = false;

// Valorile FALSY — se comporta ca false in context boolean
var_dump((bool) 0);           // bool(false)
var_dump((bool) "");         // bool(false) — string gol
var_dump((bool) "0");        // bool(false) — special in PHP!
var_dump((bool) []);         // bool(false) — array gol
var_dump((bool) null);       // bool(false)

// Valorile TRUTHY — orice altceva e true
var_dump((bool) 1);           // bool(true)
var_dump((bool) -1);         // bool(true) — orice nr nenul!
var_dump((bool) "false");    // bool(true) — string nenul!
var_dump((bool) [0]);        // bool(true) — array cu elemente

// Echo pe bool: true → '1', false → '' (string gol)
echo true;    // 1
echo false;   // (nimic afisata!)
var_dump(true); // bool(true) ← corect pentru debug
```

>> OUTPUT:

```
bool(false)
bool(false)
bool(false) ← ATENTIE: '0' e falsy, nu '1'!
bool(false)
bool(false)
bool(true)
bool(true) ← -1 e truthy!
bool(true) ← "false" ca string e truthy!
bool(true)
1          ← echo true afiseaza '1'
          ← echo false nu afiseaza nimic
```

Tipul array

Array-urile in PHP sunt structuri de date ordonate care pot contine orice tip de valori (mixt). Cheile pot fi intregi sau stringuri.

```
// --- ARRAY INDEXAT NUMERIC (indici 0, 1, 2, ...) ---
$note = [8, 9, 7, 10, 6];           // sintaxa moderna PHP 5.4+
$note2 = array(8, 9, 7, 10, 6);     // sintaxa clasica (identica)
echo $note[0];                     // 8   (primul element)
echo $note[4];                     // 6   (ultimul element)
echo $note[-1];                    // EROARE - PHP nu suporta indici negativi direct
// Adaugare elemente
$note[] = 9;                       // adauga 9 la sfarsit, index devine 5
array_push($note, 10, 8);          // adauga mai multe
// --- ARRAY ASOCIATIV (chei personalizate) ---
$student = [
    "nume"    => "Ion Popescu",
    "grupa"   => "3A",
    "medie"   => 8.75,
    "promovat" => true,
    "note"    => [9, 8, 10, 7]    // array imbricat!];
echo $student["nume"];              // Ion Popescu
echo $student["note"][2];           // 10 (al 3-lea element din note)

// --- ARRAY MIXT (chei int si string amestecate) ---
$mixt = [
    0    => "zero",
    "a"  => "litera a",
    1    => "unu",
    "b"  => "litera b",];

// Parcurgere cu foreach
foreach ($mixt as $cheie => $valoare) {
    echo "$cheie: $valoare\n";}
```

>> OUTPUT:

```
8
6
Ion Popescu
10
0: zero
a: litera a
1: unu
b: litera b
```

Tipul array

Exercitiu 2.2 — Operatii esentiale cu array-uri

Cod PHP

```
<?php
$culori = ["rosu", "verde", "albastru"];

// Numar elemente
echo count($culori) . "\n";           // 3

// Adaugare la sfarsit
$culori[] = "galben";
echo count($culori) . "\n";           // 4

// Elimina ultimul
$eliminat = array_pop($culori);
echo $eliminat . "\n";                // galben

// Sortare alfabetica
sort($culori);
print_r($culori);

// Cauta o valoare
var_dump(in_array("verde", $culori)); // true
?>
```

Output in browser

```
3
4
galben
Array
( [0] => albastru
  [1] => rosu
  [2] => verde )
bool(true)
```

Nota: sort() reorganizeaza indicii de la 0. print_r() afiseaza structura completa a unui array.

Tipuri Speciale: NULL, Resource si Rezumat

```
// NULL — absenta valorii
$x = null;
var_dump($x);           // NULL
var_dump(isset($x));    // bool(false) — variabila 'nu exista' dpdv isset

// O variabila nedeclarata are valoarea null
var_dump(isset($nedeclarata)); // bool(false)

// Verificare null
if ($x === null) echo 'Este null';
if (is_null($x))  echo 'Este null';

// RESOURCE — referinta la o resursa externa
$ fisier = fopen("date.txt", "r"); // deschide fisier pentru citire
var_dump($fisier);                // resource(3) of type (stream)
fclose($fisier);                  // intotdeauna inchide resursele!
```

Rezumat tipuri de date PHP

integer — numere intregi: 0, 42, -17, 0xFF
float — numere reale: 3.14, -2.5, 6.022e23
string — siruri: 'text', "sir cu \$variabila"
boolean — logic: true / false
array — colectii: [1, 2, 3] sau ['cheie' => 'valoare']
object — instante de clase: new MyClass()
null — absenta valorii: null
resource — referinte externe: fopen(), mysqli_connect()

Conversii de Tip (Type Casting si Type Juggling)

```
// PHP face conversii AUTOMATE (type juggling)
echo "5" + 3;           // 8 - string "5" convertit la int
echo "5.5" + 1;         // 6.5 - string "5.5" convertit la float
echo "3 mere" + 2;      // 5 - "3 mere" → 3 (pana la primul non-cifra)
echo "mere" + 2;        // 2 - "mere" → 0

// Conversie EXPLICITA (type casting) - recomandata pentru claritate
$nr = (int) "42px";      // 42
$pi = (float) "3.14abc"; // 3.14
$ok = (bool) 1;          // true
$sir = (string) 3.14;    // "3.14"
$arr = (array) "valoare"; // ["valoare"]

// Functii de conversie
intval('42px');          // 42
floatval('3.14abc');     // 3.14
strval(42);              // '42'
boolval(0);              // false

// Verificare tip
gettype(42);             // 'integer'
gettype('text');         // 'string'
is_int(42);              // true
is_string('text');       // true
is_numeric('42.5');      // true - numeric string
is_numeric('42abc');     // false
```

>> OUTPUT:

```
8
6.5
5
2
42 (cast la int)
3.14 (cast la float)
```

Variabile — Declarare si Asignare

Declarare si asignare

In PHP nu exista declarare separata de asignare. O variabila este creata in momentul primei asignari de valoare.

```
// Asignare simpla — tipul e determinat din valoare
$varsta = 20;
$prenume = "Maria";
$medie = 8.5;
$este_logat = false;

// Asignare multipla pe acelasi rand
$a = $b = $c = 0; // toti au valoarea 0

// Asignare cu operatii
$x = 5;
$y = $x + 3; // y = 8
$z = $x * $y; // z = 40

// Asignare prin referinta (&) — alias la aceeași locație de memorie
$original = 100;
$alias = &$original; // $alias si $original sunt ACELASI spatiu de memorie
$alias = 200;
echo $original; // 200 — s-a modificat si original!

// Stergere variabila
unset($alias); // sterge variabila din memorie
echo $original; // 200 — originalul ramane intact

// Verificare existenta variabila
isset($prenume); // true — variabila exista si nu e null
isset($prenume, $varsta); // true — ambele exista
empty($prenume); // false — are valoare nenula
empty(""); // true — string gol e 'empty'
```

Domeniu de Vizibilitate (Scope)

PHP are scope EXPLICIT — variabilele globale nu sunt accesibile automat in functii!

```
$global = 'Sunt variabila globala';

function demonstreazaScope() {
    // EROARE: $global nu e accesibila direct in functie!
    // echo $global; → Notice: Undefined variable $global

    // Metoda 1: keyword 'global'
    global $global;
    echo $global . "\n";    // functioneaza acum

    // Metoda 2: variabila superglobala $GLOBALS
    echo $GLOBALS['global'] . "\n";    // identic

    // Variabila locala — exista DOAR in aceasta functie
    $locala = 'Sunt locala';    echo $locala . "\n";}

demonstreazaScope();
// echo $locala; → EROARE — $locala nu exista in scope global

// --- VARIABILA STATICA --- isi pastreaza valoarea intre apeluri
function contor() {
    static $n = 0;    // initializata DOAR la primul apel
    $n++;
    echo "Apel #${n}\n";
}
contor();    // Apel #1
contor();    // Apel #2
contor();    // Apel #3
```

>> OUTPUT:

```
Sunt variabila globala
Sunt variabila globala
Sunt locala
Apel #1
Apel #2
Apel #3
```

Variabile Predefinite (Superglobale)

Disponibile in orice punct al codului, fara keyword global:

```
// $_GET - date din URL (?cheie=valoare&alta=val2)
// URL: http://localhost/test.php?id=5&actiune=editeaza
echo $_GET["id"];           // 5
echo $_GET["actiune"];      // editeaza

// $_POST - date trimise prin formular (metoda POST)
// Formular HTML cu method='post'
echo $_POST["email"];       // adresa@email.com
echo $_POST["parola"];      // parola_utilizatorului

// $_SERVER - informatii despre server si cerere
echo $_SERVER["PHP_SELF"];   // /test.php
echo $_SERVER["SERVER_NAME"]; // localhost
echo $_SERVER["REQUEST_METHOD"]; // GET sau POST
echo $_SERVER["REMOTE_ADDR"]; // IP-ul vizitatorului
echo $_SERVER["HTTP_USER_AGENT"]; // browser-ul vizitatorului

// $_SESSION - date de sesiune (persistente pe mai multe pagini)
session_start();           // TREBUIE apelat inainte de orice output
$_SESSION["utilizator"] = "Ion";
$_SESSION["rol"]         = "admin";

// $_COOKIE - date stocate in browser
setcookie('tema', 'dark', time() + 3600 * 24 * 30); // valid 30 zile
echo $_COOKIE["tema"];    // dark (disponibil la urmatoarea cerere)

// $_FILES - fisiere incarcate prin upload
echo $_FILES["poza"]["name"]; // foto.jpg
echo $_FILES["poza"]["size"];  // 204800 (bytes)
echo $_FILES["poza"]["type"];  // image/jpeg
```

Constante — define() si const

Constantele sunt valori denumite care NU se pot modifica dupa definire. Sunt accesibile global, fara \$ si fara cuvantul cheie global.

```
// Sintaxa: define('NUME_CONSTANTA', valoare);
// Conventie: UPPERCASE cu underscore
define("PI", 3.14159265358979);
define("APP_NAME", "MyApp v2.0");
define("MAX_NOTE", 10);
define("DB_HOST", "localhost");
define("DB_NAME", "studenti_db");
define("DEBUG_MODE", false);

// Constante de tip array (PHP 7+)
define("ZILE_LUCRATOARE", ["Luni", "Marti", "Miercuri", "Joi", "Vineri"]);
define("STATUSURI", ["activ" => 1, "inactiv" => 0, "blocat" => -1]);

// Utilizare - fara $ !
echo PI; // 3.14159265358979
echo APP_NAME; // MyApp v2.0
echo ZILE_LUCRATOARE[0]; // Luni
echo STATUSURI['activ']; // 1

// Verificare existenta constanta
var_dump(defined('PI')); // bool(true)
var_dump(defined('INEXISTENT')); // bool(false)

// define() permite conditii (runtime)
if (PHP_OS_FAMILY === 'Windows') {
    define("SEPARATOR", "\\");
} else {
    define("SEPARATOR", "/");
}
```

Constante — define() si const

Constantele sunt valori denumite care NU se pot modifica dupa definire. Sunt accesibile global, fara \$ si fara cuvantul cheie global.

const — constante la compile-time (in clase!):

```
// const se evalueaza la compile-time
// Se poate folosi in scope global si in clase
const VERSIUNE      = '2.5.1';
const MAX_INCERCARI = 3;
const TIMEOUT       = 30.0;

// DIFERENTA FUNDAMENTALA fata de define():
// const NU poate fi in blocuri conditionale sau functii!
if (true) {
    define('OK_IN_IF', 1);      // VALID
    // const INVALID_IN_IF = 1; // EROARE sintaxa
}

// const POATE fi folosit in clase (recomandat acolo)
class Config {
    const DB_HOST      = 'localhost';
    const DB_PORT      = 3306;
    const CHARSET      = 'utf8mb4';
}
echo Config::DB_HOST;    // localhost
echo Config::DB_PORT;    // 3306
```

Constante Predefinite PHP

```
// Versiune si sistem
echo PHP_VERSION;           // '8.3.4'
echo PHP_MAJOR_VERSION;     // 8
echo PHP_OS;                // 'Linux' sau 'WINNT'
echo PHP_OS_FAMILY;         // 'Linux', 'Windows', 'Darwin'
echo PHP_SAPI;              // 'cli', 'apache2handler', etc.

// Limite numerice
echo PHP_INT_MAX;           // 9223372036854775807
echo PHP_INT_MIN;          // -9223372036854775808
echo PHP_FLOAT_MAX;        // 1.7976931348623E+308
echo PHP_FLOAT_EPSILON;    // 2.2204460492503E-16

// Constante speciale (se schimba cu contextul)
echo __FILE__;              // /var/www/html/test.php (calea completa)
echo __DIR__;               // /var/www/html (directorul)
echo __LINE__;              // 42 (numarul liniei curente)
echo __FUNCTION__;          // 'numeleFunctiei' (in interiorul unei functii)
echo __CLASS__;             // 'NumeleClase' (in interiorul unei clase)

// Altele utile
echo PHP_EOL;               // \n (Linux) sau \r\n (Windows)
echo DIRECTORY_SEPARATOR;   // / (Linux) sau \ (Windows)
echo M_PI;                  // 3.1415926535898
echo M_E;                   // 2.718281828...
echo INF;                   // INF (infinit)
echo NAN;                   // NAN (Not a Number)
```

Operatori PHP

A) Aritmetici — +, -, *, /, %, **, intval()

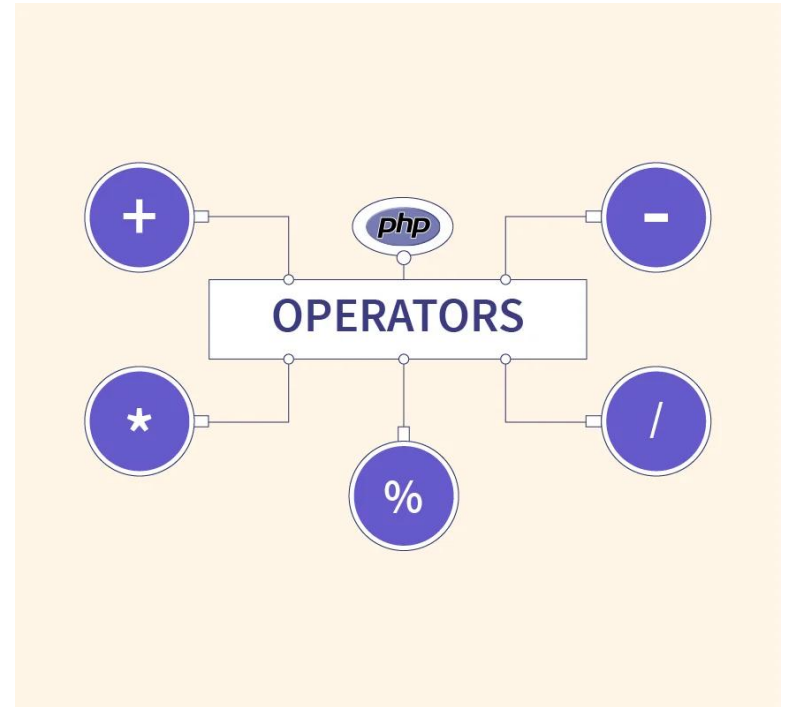
B) Asignare — =, +=, -=, *=, /=, ., ??=, ++, --

C) Comparatie — ==, ===, !=, !==, <, >, <=>

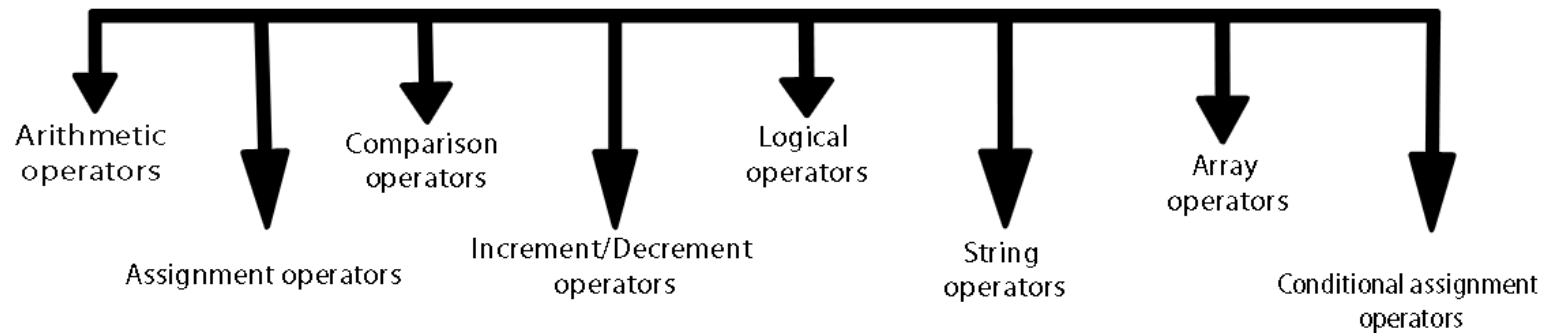
D) Logici — &&, ||, !, and, or, xor

E) Speciali — ?:, ?: (Elvis), ??, ?->

F) Bitwise — &, |, ^, ~, <<, >>



PHP Operators



Operatori Aritmetici

```
$a = 17;
$b = 5;

echo $a + $b;          // 22  - adunare
echo $a - $b;          // 12  - scadere
echo $a * $b;          // 85  - inmultire
echo $a / $b;          // 3.4 - impartire (rezultat float daca nu e exact)
echo $a % $b;          // 2   - modulo (rest la impartire)
echo $a ** $b;         // 1419857 - ridicare la putere (2^10=1024)
echo intdiv($a, $b);   // 3    - impartire intreaga (PHP 7+)

// Modulo e util pentru:
// - verificare paritate: $n % 2 === 0 → numar par
// - ciclism: $index % $lungime → indice circular
// - ultimele cifre: $nr % 100 → ultimele 2 cifre
```

Operatori de Asignare

```
$x = 10;

$x += 5;      // $x = $x + 5 → 15
$x -= 3;      // $x = $x - 3 → 12
$x *= 2;      // $x = $x * 2 → 24
$x /= 6;      // $x = $x / 6 → 4
$x %= 3;      // $x = $x % 3 → 1
$x **= 4;     // $x = $x ** 4 → 1

// Concatenare cu asignare
$mesaj = "Buna";
$mesaj .= ", lume!";    // $mesaj = $mesaj . ", lume!" → "Buna, lume!"

// Null coalescing assignment (PHP 7.4+)
$config["timeout"] ??= 30; // seteaza 30 DOAR daca e null/nedefinit",

// Incrementare / Decrementare
$i = 5;
echo ++$i;    // 6  - PRE-increment: intai creste, poi returneaza
echo $i++;    // 6  - POST-increment: intai returneaza, poi creste
echo $i;      // 7
echo --$i;    // 6  - PRE-decrement
echo $i--;    // 6  - POST-decrement
echo $i;      // 5
```

Operatori de Comparatie

```
// Egalitate LAXA (==) - cu conversie de tip
var_dump(5 == 5);          // bool(true)    - evident
var_dump(5 == "5");        // bool(true)    - "5" convertit la int
var_dump(0 == false);      // bool(true)    - 0 e falsy
var_dump("" == false);     // bool(true)    - "" e falsy
var_dump("" == null);      // bool(true)    - ambele sunt falsy
var_dump(0 == null);       // bool(true)    - ambele falsy
// Egalitate STRICTA (===) - tip SI valoare trebuie sa coincidă
var_dump(5 === 5);         // bool(true)
var_dump(5 === "5");       // bool(false)   - int vs string
var_dump(0 === false);     // bool(false)   - int vs bool
var_dump("" === null);     // bool(false)   - string vs null
// REGULA DE AUR: FOLOSITI INTOTDEAUNA ===
// Comparatii ordinale
var_dump(5 > 3);            // bool(true)
var_dump(5 >= 5);           // bool(true)
var_dump(3 < 5);            // bool(true)
var_dump(5 <= 5);           // bool(true)
var_dump(5 != 3);           // bool(true)    - diferit (lax)
var_dump(5 !== '5');        // bool(true)    - non-identic (strict)

// Operatorul spaceship <=> (PHP 7+) - returneaza -1, 0, 1
echo (1 <=> 2);              // -1   (stanga mai mica)
echo (2 <=> 2);              // 0    (egale)
echo (3 <=> 2);              // 1    (stanga mai mare)
// Util pentru sortari personalizate cu usort()
```

!! ATENTIE / EROARE FRECVENTA:

```
// Capcana clasica cu == in PHP
var_dump("php" == 0);      // bool(true) in PHP < 8 !
// In PHP < 8: stringul non-numeric e convertit la 0
// In PHP 8: string vs int → intul e convertit la string, deci false
```

```
// Alta capcana: comparatii cu null
var_dump(0 == null);        // bool(true)    - amandoua falsy
var_dump("" == null);       // bool(true)
// SOLUTIA: folositi === care verifica si tipul
```

Operatori Logici

```
// && (AND) – ambii trebuie sa fie true
$varsta = 20;
$sare_bilet = true;
var_dump($varsta >= 18 && $sare_bilet);    // bool(true)
var_dump($varsta >= 18 && !$sare_bilet);    // bool(false)

// || (OR) – cel putin unul trebuie sa fie true
$e_admin = false;
$e_moderator = true;
var_dump($e_admin || $e_moderator);    // bool(true)

// ! (NOT) – negatie
$activ = false;
var_dump(!$activ);    // bool(true)

// Short-circuit evaluation (evaluare scurta-circuitare)
// PHP NU evalueaza a doua operanda daca prima e suficienta

// Cu &&: daca primul e false → al doilea NU e evaluat
function returneazaFalse() {
    echo 'Functia a fost apelata';
    return false;
}
false && returneazaFalse();    // returneazaFalse() NU e apelata!
true || returneazaFalse();    // idem – || cu true la stanga

// Aplicatie practica: protectie la null
$user = null;
if ($user !== null && $user->esteActiv()) {
    // daca $user e null, esteActiv() NU e apelata
}

// and / or / xor – variante cu prioritate mai mica (rare in codul modern)
var_dump(true xor true);    // bool(false) – SAU exclusiv
var_dump(true xor false);    // bool(true)
```

Operatori Speciali PHP

```
// --- OPERATORUL TERNAR ? : ---
$nota = 7;
$rezultat = ($nota >= 5) ? "Admis" : "Respins";
echo $rezultat;    // "Admis"

// Poate fi imbricat (evitati mai mult de 2 niveluri!)
$calificativ = ($nota >= 9) ? "Exceptional" : (($nota >= 7) ? "Bine" : "Satisfacator");

// --- OPERATORUL ELVIS ?: ---
// Returneaza operandul stang DACA e truthy, altfel operandul drept
$input = "";
$valoare = $input ?: "valoare implicita";
echo $valoare;    // "valoare implicita" (input e falsy)

$input2 = "text";
$valoare2 = $input2 ?: "default";
echo $valoare2;    // "text" (input2 e truthy)

// --- OPERATORUL NULL COALESCING ?? --- (PHP 7+)
// Returneaza stanga DACA exista si nu e null, altfel dreapta
$user_id = $_GET["id"] ?? "anonim";
// Daca $_GET['id'] nu exista → 'anonim'
// Daca $_GET['id'] = 0 → 0 (0 e valid! diferit de ?:)

// Inlantuire cu ??
$config["timeout"] ?? $default_timeout ?? 30;

// --- OPERATORUL NULLSAFE ?-> --- (PHP 8+)
// Apeleaza metoda/proprietatea DOAR daca obiectul nu e null
$user = null;
$oras = $user?->adresa?->oras;    // null (fara eroare!)
// Fara ?->: Fatal error: Call to member function on null

// --- OPERATORUL DE SUPRESIE ERORI @ --- (EVITATI!)
// Suprima orice eroare PHP — face debugging imposibil
$x = @file_get_contents("fisier_inexistent.txt");    // Nu afiseaza eroare
// Solutia corecta: verificati cu file_exists() inainte
```

Operatori pe Biti (Bitwise)

```
// Mai putin frecventi, dar utili in sistemele embedded / flags
$a = 0b1100;    // 12 in baza 10
$b = 0b1010;    // 10 in baza 10

echo decbin($a & $b);    // 1000  (8)  - AND pe biti
echo decbin($a | $b);    // 1110  (14) - OR pe biti
echo decbin($a ^ $b);    // 0110  (6)  - XOR pe biti
echo decbin(~$a);        // ...11110011 - NOT pe biti
echo $a << 1;            // 24    - shift stanga (x2)
echo $a >> 1;            // 6     - shift dreapta (impartire 2)

// Aplicatie practica: sistem de permisiuni cu flags
define('PERM_READ',    0b0001);    // 1
define('PERM_WRITE',   0b0010);    // 2
define('PERM_EXECUTE', 0b0100);    // 4
define('PERM_DELETE',  0b1000);    // 8

$permisiuni_user = PERM_READ | PERM_WRITE;    // 3 (are read si
write)

// Verificare permisiune
if ($permisiuni_user & PERM_READ) echo "Poate citi\n";
if ($permisiuni_user & PERM_WRITE) echo "Poate scrie\n";
if ($permisiuni_user & PERM_DELETE) echo "Poate sterge\n"; // NU
```

>> OUTPUT:

```
Poate citi
Poate scrie
(PERM_DELETE nu e afisat - userul nu are aceasta permisiune)
```

Exercitiu 3.1 — Calculator cu toti operatorii

Cod PHP

```
<?php
$a = 15; $b = 4;
echo "a=$a, b=$b\n";
echo "a+b=" . ($a+$b) . "\n";      // 19
echo "a-b=" . ($a-$b) . "\n";      // 11
echo "a*b=" . ($a*$b) . "\n";      // 60
echo "a/b=" . ($a/$b) . "\n";      // 3.75
echo "a%b=" . ($a%$b) . "\n";      // 3
echo "a**b=" . ($a**$b) . "\n";     // 50625
echo "intdiv=" . intdiv($a,$b) . "\n"; // 3

// Ternary
$r = ($a > $b) ? "a mai mare" : "b mai mare";
echo $r . "\n";

// Null coalescing
$val = null ?? $a ?? 0;
echo "val=$val\n";    // 15
?>
```

Output in browser

```
a=15, b=4
a+b=19
a-b=11
a*b=60
a/b=3.75
a%b=3
a**b=50625
intdiv=3
a mai mare
val=15
```

Instructiuni Conditionale si Functii

Instructiunea if / elseif / else

Instructiunea if permite executia conditionata a blocurilor de cod. Este fundamentul oricarei logici de decizie in programare.

```
// Forma completa
if (conditie_1) {
    // executat DOAR daca conditie_1 este true
} elseif (conditie_2) {
    // executat DOAR daca conditie_1 e false si conditie_2 e true
} elseif (conditie_3) {
    // pot exista ORICATE ramuri elseif
} else {
    // executat daca NICIO conditie de mai sus nu e true
}

// IMPORTANT: Acoladele sunt RECOMANDATE chiar si pentru o singura
linie
// Forma fara acolade (permisa dar riscanta):
if ($x > 0) echo 'Pozitiv';    // OK, dar greu de extins

// Forma alternativa pentru template-uri HTML:
if ($logat): ?>
    <p>Bun venit!</p>
<?php elseif ($invitat): ?>
    <p>Vizitator</p>
<?php else: ?>
    <p>Te rog autentifica-te</p>
<?php endif; ?>
```


Exercitiu: Clasificare Nota cu if/elseif/else

Exercitiu 4.1 — Clasificare nota cu if/elseif/else

Cod PHP

```
<?php
$nota = 8.5;

if ($nota > 10 || $nota < 1) {
    echo "Nota invalida!";
} elseif ($nota >= 9.5) {
    $clas = "10 - Exceptional";
} elseif ($nota >= 8.5) {
    $clas = "9 - Foarte bine";
} elseif ($nota >= 7.5) {
    $clas = "8 - Bine";
} elseif ($nota >= 6.5) {
    $clas = "7 - Satisfacator";
} elseif ($nota >= 5.0) {
    $clas = "6 - Suficient";
} else {
    $clas = "Respins";
}

if (isset($clas)) {
    echo "Nota $nota: $clas";
}
?>
```

Output in browser

Nota 8.5: 9 - Foarte bine

Nota: Verificam intai invaliditatea
notei cu || (OR). isset() verifica
daca \$clas a fost setata

Exercitiu: Verificare an bisect cu conditii imbricate

Exercitiu 4.2 — Verificare an bisect cu conditii imbricate

Cod PHP

```
<?php
function esteBisect(int $an): bool {
    // Un an e bisect daca:
    // - e divizibil cu 4 SI (nu cu 100 SAU cu 400)
    if ($an % 400 === 0) {
        return true;        // 400 → bisect
    } elseif ($an % 100 === 0) {
        return false;       // 100 dar nu 400 → NU bisect
    } elseif ($an % 4 === 0) {
        return true;        // 4 dar nu 100 → bisect
    }
    return false;
}

$ani = [2000, 1900, 2024, 2023, 2100];
foreach ($ani as $an) {
    $b = esteBisect($an) ? "BISECT" : "normal";
    echo "$an: $b\n";
}
?>
```

Output in browser

```
2000: BISECT    (div
400)
1900: normal    (div
100 dar nu 400)
2024: BISECT    (div 4
dar nu 100)
2023: normal
2100: normal    (div
100 dar nu 400)
```

Instructiunea switch

switch evalueaza o expresie o singura data si o compara cu valorile din case. Foloseste comparatie laxa (==).

```
// Sintaxa switch
switch (expresie) {
    case valoare1:
        // cod
        break;           // OBLIGATORIU pentru a opri fall-through
    case valoare2:
        // cod
        break;
    default:             // optional – executat daca niciun case nu
se potriveste
        // cod}
```

Exercitiu 4.3 — Router simplu de pagini cu switch

Cod PHP

```
<?php
$pagina = $_GET["p"] ?? "acasa";

switch ($pagina) {
    case "acasa":
        echo "<h1>Pagina de start</h1>";
        break;
    case "despre":
        echo "<h1>Despre noi</h1>";
        break;
    case "contact":
        echo "<h1>Contactati-ne</h1>";
        break;
    case "login":
    case "autentificare": // ambele fac acelasi lucru
        echo "<h1>Autentificare</h1>";
        break;
    default:
        http_response_code(404);
        echo "<h1>Pagina nu a fost gasita</h1>";
}
?>

// Test cu URL: test.php?p=despre
```

Output in browser

```
<h1>Despre noi</h1>
(pentru ?p=despre)
<h1>Pagina de
start</h1> (pentru
?p=acasa sau fara
parametru)
<h1>404</h1>
(pentru ?p=altceva)
```

Nota: Doua case-uri consecutive fara break → ambele executa acelasi cod (fall-through intentionat)

Expresia match (PHP 8.0+)

match este alternativa moderna la switch. Avantaje: comparatie STRICTA (===), returneaza o valoare, nu are fall-through, mai concis.

```
// Sintaxa match
$rezultat = match (expresie) {
    valoare1 => expresie_de_returnat1,
    valoare2, valoare3 => expresie_de_returnat2, // mai multe valori
    default => expresie_default, // OBLIGATORIU daca nu acoperi toate
    cazurile
};
```

match(true) — evalueaza conditii complexe:

```
$scalificativ = match(true) {
    $medie >= 9.5 => 'Excelent',
    $medie >= 8.0 => 'Foarte bine',
    $medie >= 7.0 => 'Bine',
    $medie >= 5.0 => 'Suficient',
    default      => 'Respins',
};
```

Exercitiu 4.4 — Status HTTP cu match

Cod PHP

```
<?php
function descrieCodHTTP(int $cod): string {
    return match($cod) {
        200 => "OK — Cerere reusita",
        201 => "Created — Resursa creata",
        204 => "No Content — Fara continut",
        301 => "Moved Permanently",
        302 => "Found — Redirect temporar",
        400 => "Bad Request — Cerere invalida",
        401 => "Unauthorized — Autentificare necesara",
        403 => "Forbidden — Acces interzis",
        404 => "Not Found — Pagina inexistentă",
        500 => "Internal Server Error",
        502 => "Bad Gateway",
        503 => "Service Unavailable",
        default => "Cod HTTP necunoscut: $cod",
    };
}

$coduri = [200, 404, 500, 302, 999];
foreach ($coduri as $cod) {
    echo "$cod: " . descrieCodHTTP($cod) . "\n";
}
?>
```

Output in browser

```
200: OK — Cerere
reusita
404: Not Found —
Pagina inexistentă
500: Internal
Server Error
302: Found —
Redirect temporar
999: Cod HTTP
necunoscut: 999
```

Exercitiu 4.5 — match cu expresii conditionale

Cod PHP

```
<?php
// match(true) — evalueaza conditii, nu valori
function clasificaMedie(float $medie): string {
    return match(true) {
        $medie >= 9.5          => "Excelent",
        $medie >= 8.0          => "Foarte bine",
        $medie >= 7.0          => "Bine",
        $medie >= 5.0          => "Suficient",
        $medie >= 0.0          => "Respins",
        default                => "Medie
invalida",
    };
}

$medii = [9.8, 8.3, 7.1, 4.9, 5.0, -1.0];
foreach ($medii as $m) {
    echo "Medie $m → " . clasificaMedie($m) . "\n";
}
?>
```

Output in browser

```
Medie 9.8 → Excelent
Medie 8.3 → Foarte bine
Medie 7.1 → Bine
Medie 4.9 → Respins
Medie 5.0 → Suficient
Medie -1.0 → Medie
invalida
```

Nota: match(true) este o tehnica puternica — permite conditii complexe in locul valorilor simple

Funcții — Declarare și Apel

O funcție este un bloc de cod reutilizabil, cu un nume. Principiul DRY: Don't Repeat Yourself — scrii o dată, folosești oriunde.

Declararea și apelarea funcțiilor

```
// Declarare
function numeleFuncției(parametru1, parametru2) {
    // corpul funcției
    // ...
    return valoare;    // optional
}

// Apel
$rezultat = numeleFuncției(argument1, argument2);

// IMPORTANT: În PHP, funcțiile pot fi apelate ÎNAINTE de
// declarare
// (PHP hoistează funcțiile)
echo salut('Ion');    // funcționează chiar dacă funcția e
// declarată după

function salut(string $nume): string {
    return "Bună, $nume!";
}
```


Exercitiu 4.6 — Primele functii simple

Cod PHP

```
<?php
function factorial(int $n): int {
    if ($n <= 1) return 1;
    return $n * factorial($n - 1); // recursivitate
}

function sumaDigite(int $n): int {
    $n = abs($n); // ignora semnul
    $suma = 0;
    while ($n > 0) {
        $suma += $n % 10; // ultima cifra
        $n = intdiv($n, 10); // sterge ultima cifra
    }
    return $suma;
}

// Test factorial
for ($i = 0; $i <= 7; $i++) {
    echo "$i! = " . factorial($i) . "\n";
}

// Test suma cifre
$numere = [123, 9999, 1001, 7];
foreach ($numere as $nr) {
    echo "Suma cifre($nr) = " . sumaDigite($nr) . "\n";
}
?>
```

Output in browser

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
Suma cifre(123) = 6
Suma cifre(9999) = 36
Suma cifre(1001) = 2
Suma cifre(7) = 7
```

Parametri cu Valori Implicite + Named Arguments (argumente numite)

```
// Parametrii cu valori implicite
TREBUIE sa fie dupa cei fara
function formatNota(float $nota, int
$zecimale = 2, string $prefix =
'Nota:'): string {
    return $prefix . " " .
number_format($nota, $zecimale, ",",
"." );
}

echo formatNota(8.756789);           //
'Nota: 8,76'
echo formatNota(8.756789, 4);        //
'Nota: 8,7568'
echo formatNota(8.756789, 1, 'Medie:');
// 'Medie: 8,8'
```

>> OUTPUT:

```
Nota: 8,76
Nota: 8,7568
Medie: 8,8
```

```
// Putem specifica argumentele dupa NUME, nu dupa
pozitie
function creazaStudent(string $nume, int $varsta,
string $grupa = '1A'): string {
    return "$nume (varsta: $varsta, grupa: $grupa)";
}

// Apel clasic (positional)
echo creazaStudent('Ana', 20, '3B');

// Apel cu named arguments – ordinea nu mai conteaza
echo creazaStudent(varsta: 22, grupa: '3A', nume:
'Ion');

// Mixt: positional + named (named-le vin dupa
positional-e)
echo creazaStudent('Maria', grupa: '2C', varsta: 21);

// Util pentru functii built-in cu multi parametri
optional:
echo implode(separator: ', ', array: ['a', 'b', 'c']);
// 'a, b, c'
echo array_slice(array: [1,2,3,4,5], offset: 1,
length: 3);
```

Tipuri de parametri si de returnare

```
// PHP 7+ – type hints (hints nu erori, implicit)
function imparte(float $a, float $b): float {
    if ($b == 0) {
        throw new DivisionByZeroError("Nu se poate imparti la zero!");
    }
    return $a / $b;}

// void – functia nu returneaza nimic
function afiseazaSalut(string $nume): void {
    echo "Buna, $nume!";
    // return; ← permis, dar fara valoare
    // return 42; ← EROARE cu void}

// nullable – poate fi tipul specificat SAU null
function gasestePrenume(int $id): ?string {
    $prenume = ["Ana", "Ion", "Maria"];
    return $prenume[$id] ?? null;}
var_dump(gasestePrenume(0));    // string(3) 'Ana'
var_dump(gasestePrenume(99));  // NULL

// Union Types (PHP 8.0+) – mai multe tipuri acceptate
function proceseaza(int|string $input): string {
    if (is_int($input)) {
        return "Numarul: $input";    }
    return "Textul: $input";}
echo proceseaza(42);            // Numarul: 42
echo proceseaza("salut");       // Textul: salut

// Strict types – enforce type checking (PHP 7+)
declare(strict_types=1);
// Cu aceasta directiva, 'imparte(5, '2')' va da TypeError
```

Argumente variadice (numar variabil)

```
// Operatorul splat (...) colecteaza restul argumentelor intr-un array
function sumaVararg(int ...$numere): int {
    return array_sum($numere);
}
echo sumaVararg(1, 2, 3);           // 6
echo sumaVararg(10, 20, 30, 40);    // 100
echo sumaVararg();                  // 0

// Combinat cu parametri normali
function logMesaj(string $nivel, string ...$mesaje): void {
    $timestamp = date('H:i:s');
    foreach ($mesaje as $msg) {
        echo "[$timestamp][$nivel] $msg\n";
    }
}
logMesaj("INFO", "Server pornit", "DB conectata", "Cache initializat");

// Splat la apel - expandeaza array in argumente
$args = [3, 5, 2, 8, 1];
echo max(...$args);                // 8 - identic cu max(3, 5, 2, 8, 1)
echo min(...$args);                // 1
```

>> OUTPUT:

```
6
100
0
[14:30:00][INFO] Server pornit
[14:30:00][INFO] DB conectata
[14:30:00][INFO] Cache initializat
8
1
```

Functii anonime (Closures) si Arrow Functions

```
// Functie anonima – nu are nume, poate fi stocata in variabila
$dublu = function(int $x): int {
    return $x * 2;};
echo $dublu(5);    // 10
echo $dublu(7);    // 14
// Capturarea variabilelor din jur cu 'use'
$factor = 3;
$triplu = function(int $x) use ($factor): int {
    return $x * $factor;};
echo $triplu(5);    // 15
$factor = 10;
echo $triplu(5);    // 15 – $factor e capturat prin valoare la definire!
// Capturare prin referinta cu &
$apeluri = 0;
$contor_fn = function() use (&$apeluri): int {
    return ++$apeluri;};
echo $contor_fn();  // 1
echo $contor_fn();  // 2
echo $apeluri;      // 2 – variabila originala modificata
// ARROW FUNCTIONS (PHP 7.4+) – sintaxa concisa, captare automata
$factor = 5;
$inmultitor = fn($x) => $x * $factor;    // captureaza automat $factor
echo $inmultitor(4);    // 20
// Arrow functions sunt IDEALE cu array functions
$studenti = [
    ["nume" => "Ana",    "medie" => 9.2],
    ["nume" => "Bogdan", "medie" => 7.8],
    ["nume" => "Cristi", "medie" => 8.5],
    ["nume" => "Diana",  "medie" => 9.7],
    ["nume" => "Elena",  "medie" => 6.3],,];
// Filtrare: pastreaza doar promovati (medie >= 7)
$promovati = array_filter($studenti, fn($s) => $s["medie"] >= 7.0);
// Transformare: extrage doar numele
$nume_promovati = array_map(fn($s) => $s["nume"], $promovati);
// Sortare descrescatoare dupa medie
usort($promovati, fn($a, $b) => $b["medie"] <=> $a["medie"]);

// Reducere: suma mediilor
$suma_medii = array_reduce($studenti, fn($carry, $s) => $carry + $s["medie"], 0.0);
```

Exercitiu 4.7 — Pipeline functional cu array functions

Cod PHP

```
<?php
$studenti = [
    ["n" => "Ana",      "m" => 9.2, "gr" => "3A"],
    ["n" => "Bogdan",   "m" => 4.8, "gr" => "3B"],
    ["n" => "Cristi",   "m" => 8.5, "gr" => "3A"],
    ["n" => "Diana",    "m" => 9.7, "gr" => "3A"],
    ["n" => "Elena",    "m" => 3.9, "gr" => "3B"],
    ["n" => "Florin",   "m" => 7.2, "gr" => "3B"],
];

// 1. Filtrare: numai promovati din grupa 3A
$gr3a_ok = array_filter(
    $studenti,
    fn($s) => $s["m"] >= 5.0 && $s["gr"] === "3A"
);

// 2. Sortare descrescatoare
usort($gr3a_ok, fn($a,$b) => $b["m"] <=> $a["m"]);

// 3. Afisare clasament
$loc = 1;
foreach ($gr3a_ok as $s) {
    echo "$loc. {$s['n']} - {$s['m']}\n";
    $loc++;
}

// 4. Media grupei 3A
$gr3a = array_filter($studenti, fn($s) => $s["gr"] === "3A");
$medii = array_column(array_values($gr3a), "m");
$media = array_sum($medii) / count($medii);
echo "Media 3A: " . round($media, 2);
?>
```

Output in browser

```
1. Diana - 9.7
2. Ana - 9.2
3. Cristi - 8.5
Media 3A: 9.13
```

Nota: `array_filter()` nu recalculeaza indicii — de aceea folosim `array_values()` pentru a reindexa. `array_column()` extrage o coloana dintr-un array multidimensional.

Functii Built-in Esentiale — String

```
// Lungime si pozitii
strlen('Automatica')           // 10
strpos('PHP este bun','este')  // 4   (prima aparitie)
strrpos('abcbac','a')          // 3   (ultima aparitie)
substr_count('banana','an')    // 2   (numar aparitii)

// Transformari
strtolower('PHP 8.3')          // 'php 8.3'
strtoupper('php 8.3')          // 'PHP 8.3'
ucfirst('ion popescu')         // 'Ion popescu'
ucwords('ion popescu')         // 'Ion Popescu'
trim(' text ')                 // 'text'
ltrim(' text ')                // 'text '
rtrim(' text ')                // ' text'
str_pad('5', 3, '0', STR_PAD_LEFT) // '005'
str_repeat('ab', 3)             // 'ababab'
wordwrap('text lung...', 20)    // imparte la 20 caractere

// Cautare si inlocuire
str_replace('PHP','Python','PHP e bun') // 'Python e bun'
str_ireplace('PHP','Python','php e bun') // 'Python e bun' (case insensitive)
preg_replace('/[0-9]+/', '#', 'abc123')  // 'abc#' (regex)

// Separare si combina
explode(',', 'a,b,c,d')         // ['a','b','c','d']
explode(',', 'a,b,c', 3)        // ['a','b','c'] — limita 3 bucati
implode(' - ', ['a','b','c'])   // 'a - b - c'
str_split('PHP', 1)             // ['P','H','P']

// Formatare
sprintf('Nota: %.2f', 8.756)    // 'Nota: 8.76'
number_format(1234567.89, 2, ',', '.') // '1.234.567,89'
printf('%05d', 42)              // '00042'

// PHP 8+
str_contains('PHP 8', 'PHP')    // true
str_starts_with('PHP 8', 'PHP') // true
str_ends_with('PHP 8', '8')     // true
```

Funcții Built-in Esentiale — Array

```
// Informație și acces
count([1,2,3,4])           // 4
array_keys(['a'=>1,'b'=>2]) // ['a','b']
array_values(['a'=>1,'b'=>2]) // [1,2]
array_column($users, 'email') // extrage coloana 'email'
in_array('val', $arr)       // true/false
array_search('val', $arr)    // cheia sau false
isset($arr['cheie'])        // true dacă cheia există

// Adăugare și eliminare
array_push($arr, $val)       // adaugă la sfârșit
array_pop($arr)              // elimină și returnează ultimul
array_unshift($arr, $val)    // adaugă la început
array_shift($arr)            // elimină și returnează primul
array_splice($arr, 1, 2)     // elimină 2 elem de la poziția 1

// Combinare și transformare
array_merge($a, $b)          // unifică (renumerotează int keys)
array_combine($chei, $valori) // creează array asociativ
array_zip_key()              // (nu există – folosește array_combine)
array_flip(['a','b','c'])    // inversează chei și valori
array_reverse([1,2,3])       // [3,2,1]
array_unique([1,1,2,3,2])    // [1,2,3]
array_slice($arr, 1, 3)      // extrage 3 elemente de la index 1

// Sortare
sort($arr)                   // crescător, renumerotează
rsort($arr)                  // descrescător, renumerotează
asort($arr)                   // crescător, păstrează chei
arsort($arr)                  // descrescător, păstrează chei
ksort($arr)                   // sortează după chei crescător
krsort($arr)                  // sortează după chei descrescător
usort($arr, $fn)              // sortare custom cu funcție comparator
uasort($arr, $fn)             // sortare custom, păstrează chei

// Funcții functionale
array_map($fn, $arr)          // aplică fn pe fiecare element
array_filter($arr, $fn)       // filtrează elementele
array_reduce($arr, $fn, $init) // reduce la o singură valoare
array_walk($arr, $fn)         // iterează cu efect secundar

// Matematică
array_sum([1,2,3,4])         // 10
array_product([1,2,3,4])     // 24
max([3,1,4,1,5])             // 5
min([3,1,4,1,5])             // 1
```


Exemplul 1 — Sistem Gestiune Note Studenti

Un program complet care gestioneaza notele unui student: calculeaza medii, determina calificative si genereaza un raport detaliat.

```
<?php
declare(strict_types=1);
// ——— Functii ajutatoare ———
function calculeazaMedia(array $note): float {
    if (empty($note)) return 0.0;
    return array_sum($note) / count($note);}
function calificativ(float $medie): string {
    return match(true) {
        $medie >= 9.5 => "Exceptional (10)",
        $medie >= 8.5 => "Foarte bine (9)",
        $medie >= 7.5 => "Bine (8)",
        $medie >= 6.5 => "Satisfacator (7)",
        $medie >= 5.5 => "Suficient (6)",
        $medie >= 5.0 => "Promovat minim (5)",
        default      => "RESPINS",      };}
function generezaRaport(array $student): string {
    $nota_max = max($student['note']);
    $nota_min = min($student['note']);
    $medie = calculeazaMedia($student['note']);
    $note_str = implode(', ', $student['note']);
    $promovat = $medie >= 5.0 ? 'DA' : 'NU';
    $scalf = calificativ($medie);
    $linie = str_repeat('-', 40);
    return <<<RAPORT
$linie
RAPORT STUDENT: {$student['nume']} {$student['prenume']}
$linie
Grupa      : {$student['grupa']}
Materia    : {$student['materie']}
Note       : {$note_str}
Nota maxima: {$nota_max}
Nota minima: {$nota_min}
Media      : {$medie}
Calificativ: {$scalf}
Promovat   : {$promovat}
$linie
RAPORT;}
```

Exemplul 1 — Sistem Gestiune Note Studenti

```
// — Date studenti —————
$studenti = [
    ["nume"=>"Popescu", "prenume"=>"Ion", "grupa"=>"3A", "materie"=>"PHP", "note"=>[9,8,10,7,9]],
    ["nume"=>"Ionescu", "prenume"=>"Ana", "grupa"=>"3A", "materie"=>"PHP", "note"=>[6,5,7,4,6]],
    ["nume"=>"Georgescu", "prenume"=>"Radu", "grupa"=>"3B", "materie"=>"PHP", "note"=>[10,9,10,10,9]],];
// — Generare rapoarte individuale —————
foreach ($studenti as $student) {
    echo generezaRaport($student) . "\n\n";
}
// — Clasament general —————
$clasament = $studenti;
usort($clasament, function($a, $b) {
    $ma = calculeazaMedia($a['note']);
    $mb = calculeazaMedia($b['note']);
    return $mb <=> $ma; // descrescator});
echo "=== CLASAMENT GENERAL ===\n";
foreach ($clasament as $i => $s) {
    $m = round(calculeazaMedia($s['note']), 2);
    echo ($i+1) . ". {$s['prenume']} {$s['nume']} — medie: $m\n";}??
```

>> OUTPUT:

```
-----
RAPORT STUDENT: Ion Popescu
-----
```

```
Grupa      : 3A
Materia    : PHP
Note       : 9, 8, 10, 7, 9
Nota maxima: 10
Nota minima: 7
Media      : 8.6
Calificativ: Foarte bine (9)
Promovat   : DA
-----
```

```
... (similar pentru ceilalti studenti)
=== CLASAMENT GENERAL ===
1. Radu Georgescu — medie: 9.6
2. Ion Popescu — medie: 8.6
3. Ana Ionescu — medie: 5.6
```

Exemplul 2 — Validare Formular de Inregistrare

Un script complet pentru procesarea unui formular web cu validari multiple.

```
<?php
// ——— Functii de validare ———
function valideazaNume(string $nume): array {
    $erori = [];
    $nume = trim($nume);
    if (strlen($nume) < 2) {
        $erori[] = "Numele trebuie sa aiba minim 2 caractere";    }
    if (strlen($nume) > 50) {
        $erori[] = "Numele poate avea maxim 50 caractere";    }
    if (!preg_match('/^[a-zA-Z\s\-\-]+$/', $nume)) {
        $erori[] = "Numele poate contine doar litere, spatii si cratima";    }
    return $erori;}

function valideazaEmail(string $email): array {
    $erori = [];
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $erori[] = "Adresa de email este invalida";    }
    $domenii_interzise = ['tempmail.com', 'throwaway.email'];
    $domeniu = strtolower(substr($email, strpos($email, '@') + 1));
    if (in_array($domeniu, $domenii_interzise)) {
        $erori[] = "Nu sunt permise adrese email temporare";    }
    return $erori;}

function valideazaParola(string $parola): array {
    $erori = [];
    if (strlen($parola) < 8) {
        $erori[] = "Parola trebuie sa aiba minim 8 caractere";    }
    if (!preg_match('/[A-Z]/', $parola)) {
        $erori[] = "Parola trebuie sa contina cel putin o litera mare";    }
    if (!preg_match('/[0-9]/', $parola)) {
        $erori[] = "Parola trebuie sa contina cel putin o cifra";    }
    if (!preg_match('/[^a-zA-Z0-9]/', $parola)) {
        $erori[] = "Parola trebuie sa contina cel putin un caracter special";    }
    return $erori;}
}
```

Exemplul 2 — Validare Formular de Inregistrare

```
function valideazaVarsta(string $data_nasterii): array {
    $erori = [];
    $dn = DateTime::createFromFormat('Y-m-d', $data_nasterii);
    if (!$dn) {
        $erori[] = "Format data incorecta (folositi YYYY-MM-DD)";
        return $erori;
    }
    $azi = new DateTime();
    $varsta = $dn->diff($azi)->y;
    if ($varsta < 18) {
        $erori[] = "Trebuie sa aveti minim 18 ani";
    }
    if ($varsta > 120) {
        $erori[] = "Data nasterii invalida";
    }
    return $erori;
}

// — Procesare inregistrare —————
function proceseazaInregistrare(array $date): array {
    $toate_erorile = [];
    $erori_n = valideazaNume($date["nume"] ?? "");
    $erori_e = valideazaEmail($date["email"] ?? "");
    $erori_p = valideazaParola($date["parola"] ?? "");
    $erori_v = valideazaVarsta($date["data_nasterii"] ?? "");
    if ($erori_n) $toate_erorile["Nume"] = $erori_n;
    if ($erori_e) $toate_erorile["Email"] = $erori_e;
    if ($erori_p) $toate_erorile["Parola"] = $erori_p;
    if ($erori_v) $toate_erorile["Data nasterii"] = $erori_v;
    if (empty($toate_erorile)) {
        // Inregistrare reusita — in realitate am salva in BD
        return [
            'succes' => true,
            'utilizator' => [
                'id' => rand(1000, 9999),
                'nume' => trim($date["nume"]),
                'email' => strtolower(trim($date["email"])),
                'hash_parola' => password_hash($date["parola"], PASSWORD_BCRYPT),
                'creat_la' => date("Y-m-d H:i:s"),
            ]
        ];
    }

    return ['succes' => false, 'erori' => $toate_erorile];
}
```

```
// — Testare —
// Caz 1: Date valide
$ok = proceseazaInregistrare([
    "nume"          => "Ion Popescu",
    "email"         => "ion@example.com",
    "parola"        => "Parola123!",
    "data_nasterii" => "2000-05-15",]);
if ($ok['succes']) {
    echo "Inregistrare reusita!\n";
    echo "ID utilizator: " . $ok['utilizator']['id'] . "\n";
    echo "Email: " . $ok['utilizator']['email'] . "\n";}
// Caz 2: Date invalide
$gresit = proceseazaInregistrare([
    "nume"          => "A",
    "email"         => "invalid-email",
    "parola"        => "parola",
    "data_nasterii" => "2015-01-01",]);
if (!$gresit['succes']) {
    echo "\nErori de validare:\n";
    foreach ($gresit['erori'] as $camp => $erori) {
        echo "[$camp]:\n";
        foreach ($erori as $e) { echo "    - $e\n"; }    }}?>
```

>> OUTPUT:

```
Inregistrare reusita!
ID utilizator: 4872
Email: ion@example.com
Erori de validare:
[Nume]:
    - Numele trebuie sa aiba minim 2 caractere
[Email]:
    - Adresa de email este invalida
[Parola]:
    - Parola trebuie sa aiba minim 8 caractere
    - Parola trebuie sa contina cel putin o litera mare
    - Parola trebuie sa contina cel putin o cifra
    - Parola trebuie sa contina cel putin un caracter special
[Data nasterii]:
    - Trebuie sa aveti minim 18 ani
```

Exemplul 3 — Algoritmi Clasici in PHP

```
<?php
// — 1. Numerele prime —————
function estePrim(int $n): bool {
    if ($n < 2) return false;
    if ($n === 2) return true;
    if ($n % 2 === 0) return false;
    for ($i = 3; $i * $i <= $n; $i += 2) {
        if ($n % $i === 0) return false;    }
    return true;}
// Sirul de Eratostene – toate primele pana la N
function siruEratostene(int $n): array {
    $sita = array_fill(2, $n - 1, true);
    for ($i = 2; $i * $i <= $n; $i++) {
        if ($sita[$i]) {
            for ($j = $i * $i; $j <= $n; $j += $i) {
                $sita[$j] = false;          }          }          }
    return array_keys(array_filter($sita));}
$prime = siruEratostene(50);
echo "Prime pana la 50: " . implode(', ', $prime) . "\n";
echo "Sunt " . count($prime) . " numere prime\n\n";

// — 2. Fibonacci iterativ si recursiv —————
function fibonacciIterativ(int $n): array {
    if ($n <= 0) return [];
    if ($n === 1) return [0];
    $fib = [0, 1];
    for ($i = 2; $i < $n; $i++) {
        $fib[] = $fib[$i-1] + $fib[$i-2];    }
    return $fib;}
$fib15 = fibonacciIterativ(15);
echo "Fibonacci (15 termeni): " . implode(', ', $fib15) . "\n\n";
```

Exemplul 3 — Algoritmi Clasici in PHP

```
// — 3. Sortare —
function bubbleSort(array $arr): array {
    $n = count($arr);
    for ($i = 0; $i < $n - 1; $i++) {
        $schimbat = false;
        for ($j = 0; $j < $n - $i - 1; $j++) {
            if ($arr[$j] > $arr[$j + 1]) {
                [$arr[$j], $arr[$j + 1]] = [$arr[$j + 1], $arr[$j]];
                $schimbat = true;
            }
        }
        if (!$schimbat) break; // deja sortat
    }
    return $arr;
}

$nesortate = [64, 34, 25, 12, 22, 11, 90];
echo "Inainte: " . implode(' ', $nesortate) . "\n";
$sortate = bubbleSort($nesortate);
echo "Dupa BubbleSort: " . implode(' ', $sortate) . "\n\n";

// — 4. Cautare binara —
function cautareBinara(array $arr, int $tinta): int {
    $stanga = 0;
    $dreapta = count($arr) - 1;
    while ($stanga <= $dreapta) {
        $mijloc = intdiv($stanga + $dreapta, 2);
        if ($arr[$mijloc] === $tinta) return $mijloc;
        if ($arr[$mijloc] < $tinta) $stanga = $mijloc + 1;
        else $dreapta = $mijloc - 1;
    }
    return -1; // negasit
}

$sir_sortat = [1, 3, 5, 7, 9, 11, 13, 15, 17, 19];
echo "Cautare binara (9) in [1..19]: index " . cautareBinara($sir_sortat, 9) . "\n";
echo "Cautare binara (6) in [1..19]: index " . cautareBinara($sir_sortat, 6) . "\n";
?>
```

>> OUTPUT:

```
Prime pana la 50: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
Sunt 15 numere prime
Fibonacci (15 termeni): 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377
Inainte: 64, 34, 25, 12, 22, 11, 90
Dupa BubbleSort: 11, 12, 22, 25, 34, 64, 90
Cautare binara (9) in [1..19]: index 4
Cautare binara (6) in [1..19]: index -1 (negasit)
```

Aplicatii propuse

Aplicatia 1 — Functii matematice:

Scrieti urmatoarele functii, cu cod si output documentat:

- a) `cmmdc(a, b)` — cel mai mare divizor comun (algoritmul lui Euclid)
- b) `cmmmc(a, b)` — cel mai mic multiplu comun
- c) `estePerfect(n)` — true daca suma divizorilor proprii = n (ex: $6=1+2+3$)
- d) `descompuneFact(n)` — returneaza factorizarea primara ca array

Testati cu: `cmmdc(48,18)`, `cmmmc(4,6)`, `estePerfect(28)`, `descompuneFact(360)`

Aplicatia 2 — Prelucrare stringuri:

- a) `estePalindrom(s)` — true daca sirul e palindrom (ignorand spatii si majuscule)

Ex: "A man a plan a canal Panama" → true

- b) `numaraVocale(s)` — numara vocalele dintr-un sir

- c) `inverseazaCuvinte(s)` — inverseaza ordinea cuvintelor intr-o propozitie

Ex: "PHP este bun" → "bun este PHP"

- d) `cripteazaCezar(s, k)` — criptare Cezar cu deplasament k

Ex: `cripteazaCezar("PHP", 3)` → "SKS"

Aplicatia 3 — Prelucrare array-uri (produse cu id, pret, categorie, stoc):

Aveti un array de produse cu: id, nume, pret, categorie, stoc.

Implementati functii pentru:

- a) filtreazaDupaCategorie(produse, cat) — returneaza produsele din categoria data
- b) sorteazaDupaPret(produse, dir) — crescator sau descrescator
- c) calculeazaValoareStoc(produse) — total pret*stoc
- d) top3ScumpeProduse(produse) — primele 3 produse ca pret
- e) grupezaDupaCategorie(produse) — dict cheie=categorie, valoare=array de produse

Aplicatia 4 — Formular web complet (avansata):

Creati un formular HTML + script PHP de procesare pentru inscrierea la un concurs:

Campuri: Nume (req), Prenume (req), Email (req, unic), Telefon (format 07XXXXXXXX),
Varsta (18-35), Categorie (dropdown: Junior/Senior/Veteran),
Mesaj (optional, max 500 caractere), Accept termeni (checkbox obligatoriu)

Validari complete pe server cu afisarea erorilor langa fiecare camp

La succes: afisati un rezumat al datelor trimise

Bonus: pastrati valorile introduse in caz de eroare (nu goliti formularul).

Resurse pentru rezolvarea aplicatiilor propuse

1. Documentatia PHP oficiala (in romana partiala): <https://php.net/manual/ro>
2. PHP: The Right Way (best practices): <https://phptherightway.com>
3. Laracasts — tutoriale video de calitate: <https://laracasts.com>
4. PHP Fiddle — testati cod PHP online: <https://phpfiddle.org>
5. Composer — dependency manager: <https://getcomposer.org>
6. Laravel — framework PHP modern: <https://laravel.com>
7. PHP-FIG — standarde de cod (PSR): <https://www.php-fig.org>

